

1. To Create Main Loop, Display Prompt, Read User Input, Handle Exit Conditions, Design Control Flow Diagram, Test Interactive Loop

To Capture Keyboard Input, Handle Backspace, Process Enter Key, Manage Input Buffer, Support Multi-Character Commands, Test User Interaction

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <termios.h>
#include <unistd.h>

#define MAX 100

// Read one character without waiting for Enter
char getch() {
    struct termios oldt, newt;
    char ch;

    tcgetattr(STDIN_FILENO, &oldt);
    newt = oldt;

    newt.c_lflag &= ~(ICANON | ECHO);
    tcsetattr(STDIN_FILENO, TCSANOW, &newt);

    ch = getchar();

    tcsetattr(STDIN_FILENO, TCSANOW, &oldt);

    return ch;
}

int main() {
    char input[MAX];
    int index;

    printf("Simple Shell\n");
    printf("Type a command (type 'exit' to quit)\n");

    while (1) {
        index = 0;
        printf("\nshell> ");
        fflush(stdout);

        while (1) {
            char ch = getch();

            // Enter key
            if (ch == '\n' || ch == '\r') {
                input[index] = '\0';
                printf("\n");
                break;
            }

            // Backspace key
            else if (ch == 127 || ch == 8) {
                if (index > 0) {
                    index--;
                    printf("\b \b");
                    fflush(stdout);
                }
            }

            // Normal character
            else if (index < MAX - 1) {
                input[index++] = ch;
                printf("%c", ch);
                fflush(stdout);
            }
        }

        // Exit condition
        if (strcmp(input, "exit") == 0) {
            printf("Exiting shell...\n");
            break;
        }

        // Empty input
        if (strlen(input) == 0) {
            continue;
        }

        printf("You entered: %s\n", input);
    }

    return 0;
}
```

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <termios.h>
#include <unistd.h>

#define MAX 100

// Read one character without waiting for Enter
char getch() {
    struct termios oldt, newt;
    char ch;

    tcgetattr(STDIN_FILENO, &oldt);
    newt = oldt;

    newt.c_lflag &= ~(ICANON | ECHO);
    tcsetattr(STDIN_FILENO, TCSANOW, &newt);

    ch = getchar();

    tcsetattr(STDIN_FILENO, TCSANOW, &oldt);

    return ch;
}

int main() {
    char input[MAX];
    int index;

    printf("Simple Shell\n");
    printf("Type a command (type 'exit' to quit)\n");

    while (1) {
        index = 0;
        printf("\nshell> ");
        fflush(stdout);

        while (1) {
            char ch = getch();

            // Enter key
            if (ch == '\n' || ch == '\r') {
                input[index] = '\0';
                printf("\n");
                break;
            }
        }
    }
}
```

1. To Create Main Loop, Display Prompt, Read User Input, Handle Exit Conditions, Design Control Flow Diagram, Test Interactive Loop

To Capture Keyboard Input, Handle Backspace, Process Enter Key, Manage Input Buffer, Support Multi-Character Commands, Test User Interaction

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <termios.h>
#include <unistd.h>

#define MAX 100

// Read one character without waiting for Enter
char getch() {
    struct termios oldt, newt;
    char ch;

    tcgetattr(STDIN_FILENO, &oldt);
    newt = oldt;

    newt.c_lflag &= ~(ICANON | ECHO);
    tcsetattr(STDIN_FILENO, TCSANOW, &newt);

    ch = getchar();

    tcsetattr(STDIN_FILENO, TCSANOW, &oldt);

    return ch;
}

int main() {
    char input[MAX];
    int index;

    printf("Simple Shell\n");
    printf("Type a command (type 'exit' to quit)\n");

    while (1) {
        index = 0;
        printf("\nshell> ");
        fflush(stdout);

        while (1) {
            char ch = getch();

            // Enter key
            if (ch == '\n' || ch == '\r') {
                input[index] = '\0';
                printf("\n");
                break;
            }

            // Backspace key
            else if (ch == 127 || ch == 8) {
                if (index > 0) {
                    index--;
                    printf("\b \b");
                    fflush(stdout);
                }
            }

            // Normal character
            else if (index < MAX - 1) {
                input[index++] = ch;
                printf("%c", ch);
                fflush(stdout);
            }
        }

        // Exit condition
        if (strcmp(input, "exit") == 0) {
            printf("Exiting shell...\n");
            break;
        }

        // Empty input
        if (strlen(input) == 0) {
            continue;
        }

        printf("You entered: %s\n", input);
    }

    return 0;
}
```

1. To Create Main Loop, Display Prompt, Read User Input, Handle Exit Conditions, Design Control Flow Diagram, Test Interactive Loop

To Capture Keyboard Input, Handle Backspace, Process Enter Key, Manage Input Buffer, Support Multi-Character Commands, Test User Interaction

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <termios.h>
#include <unistd.h>

#define MAX 100

// Read one character without waiting for Enter
char getch() {
    struct termios oldt, newt;
    char ch;

    tcgetattr(STDIN_FILENO, &oldt);
    newt = oldt;

    newt.c_lflag &= ~(ICANON | ECHO);
    tcsetattr(STDIN_FILENO, TCSANOW, &newt);

    ch = getchar();
    tcsetattr(STDIN_FILENO, TCSANOW, &oldt);

    return ch;
}

int main() {
    char input[MAX];
    int index;

    printf("Simple Shell\n");
    printf("Type a command (type 'exit' to quit)\n");

    while (1) {
        index = 0;
        printf("\nshell> ");
        fflush(stdout);

        while (1) {
            char ch = getch();

            // Enter key
            if (ch == '\n' || ch == '\r') {
                input[index] = '\0';
                printf("\n");
                break;
            }

            // Backspace key
            else if (ch == 127 || ch == 8) {
                if (index > 0) {
                    index--;
                    printf("\b \b");
                    fflush(stdout);
                }
            }

            // Normal character
            else if (index < MAX - 1) {
                input[index++] = ch;
                printf("%c", ch);
                fflush(stdout);
            }
        }

        // Exit condition
        if (strcmp(input, "exit") == 0) {
            printf("Exiting shell...\n");
            break;
        }

        // Empty input
        if (strlen(input) == 0) {
            continue;
        }

        printf("You entered: %s\n", input);
    }

    return 0;
}
```

Skill_1 Skill_1.c

[ubuntu@ubuntu:~/OSLab/Skill\$./Skill_1

Simple Shell

Type a command (type 'exit' to quit)

[shell> hi

You entered: hi

[shell> ls

You entered: ls

[shell> exit

Exiting shell...